

Date: 27 / 11 / 2025

Lab Practical #1:

- 1) Write a C program to count the number of 1's and 0's in the binary representation of a number.
- 2) Write a C program to perform bitwise AND, OR, XOR, and NOT operations on integers.
- 3) Write a C program that takes a string and performs bitwise AND and XOR operations with a constant value (e.g., 127). (Simulates simple encryption idea)
- 4) Write a C program to implement left and right circular bit rotations.
- 5) Write a C program to swap two variables without using a third variable (using XOR).

Program-1:

```
#include <stdio.h>

void countAndBinary(int n)
{
    int original = n;
    int count0 = 0, count1 = 0;
    int binary[32];
    int index = 0;

    while (n > 0)
    {
        int rem = n % 2;
        binary[index++] = rem;

        if (rem == 1)
            count1++;
        else
            count0++;

        n = n / 2;
    }

    printf("Binary of %d is:", original);
    for (int i = index - 1; i >= 0; i--)
    {
        printf("%d", binary[i]);
    }

    printf("\nCount of 1s = %d\n", count1);
    printf("Count of 0s = %d\n", count0);
}

int main()
{
    int n;

    printf("Enter any Number: ");
```



Date: 27 / 11 / 2025

```
scanf("%d", &n);  
  
countAndBinary(n);  
  
return 0;  
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-1>gcc p1.c  
  
D:\CSE B-Tech\Semester-6\INS\Lab-1>a  
Enter any Number: 567  
Binary of 567 is: 1000110111  
Count of 1s = 6  
Count of 0s = 4
```

Date: 27 / 11 / 2025

Program-2:

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    printf("Bitwise AND of %d and %d is: %d\n", a, b, a & b);
    printf("Bitwise OR of %d and %d is: %d\n", a, b, a | b);
    printf("Bitwise XOR of %d and %d is: %d\n", a, b, a ^ b);
    printf("Bitwise NOT of %d is: %d\n", a, ~a);
    printf("Bitwise NOT of %d is: %d\n", b, ~b);

    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-1>gcc p2.c
D:\CSE B-Tech\Semester-6\INS\Lab-1>a
Enter two integers: 12
5
Bitwise AND of 12 and 5 is: 4
Bitwise OR of 12 and 5 is: 13
Bitwise XOR of 12 and 5 is: 9
Bitwise NOT of 12 is: -13
Bitwise NOT of 5 is: -6
```

Date: 27 / 11 / 2025

Program-3:

```
#include <stdio.h>
#include <string.h>

#define KEY 127

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    str[strcspn(str, "\n")] = '\0';

    int len = strlen(str);

    printf("\nOriginal String: %s\n", str);

    // AND
    printf("After AND with %d: ", KEY);
    for (int i = 0; i < len; i++) {
        printf("%c", str[i] & KEY);
    }
    printf("\n");

    // XOR (print hex)
    printf("After XOR with %d (hex): ", KEY);
    for (int i = 0; i < len; i++) {
        printf("%02X ", str[i] ^ KEY);
    }
    printf("\n");

    return 0;
}
```



Date: 27 / 11 / 2025

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-1>gcc p3.c

D:\CSE B-Tech\Semester-6\INS\Lab-1>a
Enter a string: darshan

Original String: darshan
After AND with 127: darshan
After XOR with 127 (hex): 1B 1E 0D 0C 17 1E 11
```



Date: 27 / 11 / 2025

Program-4:

```
#include <stdio.h>

int main() {
    unsigned int num, n, left_rotate, right_rotate;
    int bits = 32;

    printf("Enter a number: ");
    scanf("%u", &num);

    printf("Enter number of positions to rotate: ");
    scanf("%u", &n);

    n %= bits;

    left_rotate = (num << n) | (num >> (bits - n));
    right_rotate = (num >> n) | (num << (bits - n));

    printf("\nOriginal Number: %u\n", num);
    printf("After Left Circular Rotation : %u\n", left_rotate);
    printf("After Right Circular Rotation: %u\n", right_rotate);

    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-1>gcc p4.c
D:\CSE B-Tech\Semester-6\INS\Lab-1>a
Enter a number: 45
Enter number of positions to rotate: 1

Original Number: 45
After Left Circular Rotation : 90
After Right Circular Rotation: 2147483670
```

Date: 27 / 11 / 2025

Program-5:

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);

    // Swap using XOR
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;

    printf("\nAfter swapping:\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-1>gcc p5.c

D:\CSE B-Tech\Semester-6\INS\Lab-1>a
Enter value of a: 23
Enter value of b: 32

After swapping:
a = 32
b = 23
```

Date: 04 / 12 / 2025

Lab Practical #2:

- 1) Write a program to reverse a string using pointers.
- 2) Implement string functions like strlen, strcat, and strcmp.
- 3) Implement a program to read and write a text file.
- 4) Write a C program to test whether a number is prime or composite.
- 5) Write a C program to generate prime numbers in a given range.

Program-1:

```
#include <stdio.h>

int main() {
    char str[100], *ptr_start, *ptr_end, temp;
    int len = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    while (str[len] != '\0' && str[len] != '\n')
        len++;

    ptr_start = str;
    ptr_end = str + len - 1;

    while (ptr_start < ptr_end) {
        temp = *ptr_start;
        *ptr_start = *ptr_end;
        *ptr_end = temp;

        ptr_start++;
        ptr_end--;
    }

    printf("Reversed string: %s\n", str);

    return 0;
}
```




Date: 04 / 12 / 2025

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>gcc p1.c  
  
D:\CSE B-Tech\Semester-6\INS\Lab-2>a  
Enter a string: darshan  
Reversed string: nahsrad
```

Date: 04 / 12 / 2025

Program-2:

```
#include <stdio.h>

int my_strlen(char str[]) {
    int i = 0;
    while (str[i] != '\0') {
        i++;
    }
    return i;
}

void my_strcat(char dest[], char src[]) {
    int i = 0, j = 0;

    while (dest[i] != '\0') {
        i++;
    }

    while (src[j] != '\0') {
        dest[i] = src[j];
        i++;
        j++;
    }

    dest[i] = '\0';
}

int my_strcmp(char s1[], char s2[]) {
    int i = 0;

    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i]) {
            return s1[i] - s2[i];
        }
        i++;
    }

    return s1[i] - s2[i];
}

int main() {
    char s1[100], s2[100];

    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
```

Date: 04 / 12 / 2025

```
printf("Enter second string: ");
fgets(s2, sizeof(s2), stdin);

for (int i = 0; s1[i]; i++)
    if (s1[i] == '\n') s1[i] = '\0';

for (int i = 0; s2[i]; i++)
    if (s2[i] == '\n') s2[i] = '\0';

printf("\nLength of s1 : %d\n", my_strlen(s1));
printf("Length of s2 : %d\n", my_strlen(s2));

my_strcat(s1, s2);
printf("After concatenation (s1 + s2): %s\n", s1);

int result = my_strcmp(s1, s2);

if (result == 0)
    printf("Strings are equal.\n");
else if (result > 0)
    printf("s1 is greater than s2.\n");
else
    printf("s1 is smaller than s2.\n");

return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>gcc p2.c

D:\CSE B-Tech\Semester-6\INS\Lab-2>a
Enter first string: darshan
Enter second string: marwadi

Length of s1 : 7
Length of s2 : 7
After concatenation (s1 + s2): darshanmarwadi
s1 is smaller than s2.
```

Date: 04 / 12 / 2025

Program-3:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char text[200];

    fp = fopen("sample.txt", "w");

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    printf("Enter text to write into file: ");
    fgets(text, sizeof(text), stdin);

    fprintf(fp, "%s", text);
    fclose(fp);

    fp = fopen("sample.txt", "r");

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    printf("\nReading from file:\n");
    while (fgets(text, sizeof(text), fp) != NULL) {
        printf("%s", text);
    }

    fclose(fp);

    return 0;
}
```

Output:



Date: 04 / 12 / 2025

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>gcc p3.c
```

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>a  
Enter text to write into file: hello
```

```
Reading from file:  
hello
```

Date: 04 / 12 / 2025

Program-4:

```
#include <stdio.h>

int main() {
    int num, i, flag = 0;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num <= 1) {
        printf("Number is neither prime nor composite.\n");
        return 0;
    }

    // Check divisors from 2 to num/2
    for (i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            flag = 1;
            break;
        }
    }

    if (flag == 0)
        printf("%d is a PRIME number.\n", num);
    else
        printf("%d is a COMPOSITE number.\n", num);

    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>gcc p4.c
D:\CSE B-Tech\Semester-6\INS\Lab-2>a
Enter a number: 24
24 is a COMPOSITE number.
```

Date: 04 / 12 / 2025

Program-5:

```
#include <stdio.h>

int main() {
    int start, end, i, j, flag;

    printf("Enter start of range: ");
    scanf("%d", &start);

    printf("Enter end of range: ");
    scanf("%d", &end);

    printf("Prime numbers between %d and %d are:\n", start, end);

    for (i = start; i <= end; i++) {
        if (i <= 1)
            continue;

        flag = 0;

        for (j = 2; j <= i / 2; j++) {
            if (i % j == 0) {
                flag = 1;
                break;
            }
        }

        if (flag == 0)
            printf("%d ", i);
    }

    printf("\n");
    return 0;
}
```

Output:



Date: 04 / 12 / 2025

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>gcc p5.c
```

```
D:\CSE B-Tech\Semester-6\INS\Lab-2>a
```

```
Enter start of range: 34
```

```
Enter end of range: 87
```

```
Prime numbers between 34 and 87 are:
```

```
37 41 43 47 53 59 61 67 71 73 79 83
```


Date: 11 / 12 / 2025

Lab Practical #3:

Implementation of Caesar Cipher Techniques.

Program:

```
#include <stdio.h>

int main() {
    char text[100], encrypted[100], decrypted[100];
    int key, i;

    printf("Enter a message: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter key (number of shifts): ");
    scanf("%d", &key);

    for (i = 0; text[i] != '\0'; i++) {
        char ch = text[i];

        if (ch >= 'A' && ch <= 'Z')
            encrypted[i] = (ch - 'A' + key) % 26 + 'A';
        else if (ch >= 'a' && ch <= 'z')
            encrypted[i] = (ch - 'a' + key) % 26 + 'a';
        else
            encrypted[i] = ch;
    }
    encrypted[i] = '\0';

    printf("\nEncrypted Text: %s", encrypted);

    for (i = 0; encrypted[i] != '\0'; i++) {
        char ch = encrypted[i];

        if (ch >= 'A' && ch <= 'Z')
            decrypted[i] = (ch - 'A' - key + 26) % 26 + 'A';
        else if (ch >= 'a' && ch <= 'z')
            decrypted[i] = (ch - 'a' - key + 26) % 26 + 'a';
        else
            decrypted[i] = ch;
    }
    decrypted[i] = '\0';

    printf("Decrypted Text: %s", decrypted);
}
```



Date: 11 / 12 / 2025

```
return 0;  
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-3>gcc p1.c  
  
D:\CSE B-Tech\Semester-6\INS\Lab-3>a  
Enter a message: Parth  
Enter key (number of shifts): 3  
  
Encrypted Text: Sduwk  
Decrypted Text: Parth
```



Date: 20 / 12 / 2025

Lab Practical #4:

Implementation of monoalphabetic and polyalphabetic substitution cipher technique.

Program:

```
// Monoalphabetic cipher implementation in C.

#include <stdio.h>
#include <ctype.h>

int main() {
    char text[100];
    int key;

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter key: ");
    scanf("%d", &key);

    for (int i = 0; text[i] != '\0'; i++) {
        if (isalpha(text[i])) {
            char base = islower(text[i]) ? 'a' : 'A';
            text[i] = (text[i] - base + key) % 26 + base;
        }
    }

    printf("Encrypted text: %s", text);
    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-4>gcc monoAlphabetic.c
D:\CSE B-Tech\Semester-6\INS\Lab-4>a
Enter text: HELLO
Enter key: 3
Encrypted text: KH00R
```

Date: 20 / 12 / 2025

Program:

```
// Polyalphabetic Cipher Implementation in C.

#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char text[100], key[20];
    int i, j = 0;

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter key: ");
    scanf("%s", key);

    for (i = 0; text[i] != '\0'; i++) {
        if (isalpha(text[i])) {
            char base = islower(text[i]) ? 'a' : 'A';
            text[i] = (text[i] - base + (tolower(key[j]) - 'a')) % 26 + base;
            j = (j + 1) % strlen(key);
        }
    }

    printf("Encrypted text: %s", text);
    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-4>gcc polyAlphabetic.c
D:\CSE B-Tech\Semester-6\INS\Lab-4>a
Enter text: Hello
Enter key: KEY
Encrypted text: Rijvs
```

Date: 25 / 12 / 2025

Lab Practical #5:

Implementation of Playfair Cipher Technique.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char keyTable[5][5];

// Function to generate 5x5 key matrix
void generateKeyTable(char key[]) {
    int used[26] = {0};
    int i, j, k = 0;

    for (i = 0; key[i]; i++) {
        char ch = toupper(key[i]);
        if (ch == 'J') ch = 'I';
        if (!used[ch - 'A']) {
            keyTable[k / 5][k % 5] = ch;
            used[ch - 'A'] = 1;
            k++;
        }
    }

    for (i = 0; i < 26; i++) {
        if (i + 'A' == 'J') continue;
        if (!used[i]) {
            keyTable[k / 5][k % 5] = i + 'A';
            k++;
        }
    }
}

// Find position of character in key table
void findPosition(char ch, int *row, int *col) {
    int i, j;
    if (ch == 'J') ch = 'I';
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++)
            if (keyTable[i][j] == ch) {
                *row = i;
                *col = j;
            }
}
```

```
// Encrypt plaintext
void encrypt(char text[]) {
    int i, r1, c1, r2, c2;
    for (i = 0; i < strlen(text); i += 2) {
        findPosition(text[i], &r1, &c1);
        findPosition(text[i + 1], &r2, &c2);

        if (r1 == r2) {
            printf("%c%c",
                keyTable[r1][(c1 + 1) % 5],
                keyTable[r2][(c2 + 1) % 5]);
        }
        else if (c1 == c2) {
            printf("%c%c",
                keyTable[(r1 + 1) % 5][c1],
                keyTable[(r2 + 1) % 5][c2]);
        }
        else {
            printf("%c%c",
                keyTable[r1][c2],
                keyTable[r2][c1]);
        }
    }
}

int main() {
    char key[20], text[50];

    printf("Enter Key: ");
    scanf("%s", key);

    printf("Enter Plain Text (even length): ");
    scanf("%s", text);

    generateKeyTable(key);

    printf("\nEncrypted Text: ");
    encrypt(text);

    return 0;
}
```



Date: 25 / 12 / 2025

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-5>gcc playfair.c

D:\CSE B-Tech\Semester-6\INS\Lab-5>A
Enter Key: CODE
Enter Plain Text (even length): HELLO

Encrypted Text: NHMMFR
D:\CSE B-Tech\Semester-6\INS\Lab-5>|
```



Date: 09 / 01 / 2026

Lab Practical #6:

Implementation of Hill Cipher Technique.

1. 2x2 key matrix
2. 3x3 key matrix

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    int key[2][2] = {{3, 3}, {2, 5}};
    char plaintext[100];
    int i;

    printf("Enter Plaintext (uppercase): ");
    scanf("%s", plaintext);

    // If length is odd, add X
    if (strlen(plaintext) % 2 != 0) {
        strcat(plaintext, "X");
    }

    printf("Ciphertext: ");

    for (i = 0; i < strlen(plaintext); i += 2) {
        int p1 = plaintext[i] - 'A';
        int p2 = plaintext[i + 1] - 'A';

        int c1 = (key[0][0]*p1 + key[0][1]*p2) % 26;
        int c2 = (key[1][0]*p1 + key[1][1]*p2) % 26;

        printf("%c%c", c1 + 'A', c2 + 'A');
    }

    return 0;
}
```

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-6>gcc two_matrix.c
D:\CSE B-Tech\Semester-6\INS\Lab-6>a
Enter Plaintext (uppercase): OK
Ciphertext: UA
```


Date: 09 / 01 / 2026

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    int key[3][3] = {
        {6, 24, 1},
        {13, 16, 10},
        {20, 17, 15}
    };

    char plaintext[100];
    int i;

    printf("Enter Plaintext (uppercase): ");
    scanf("%s", plaintext);

    // Padding with X if needed
    while (strlen(plaintext) % 3 != 0) {
        strcat(plaintext, "X");
    }

    printf("Ciphertext: ");

    for (i = 0; i < strlen(plaintext); i += 3) {
        int p[3], c[3], j;

        for (j = 0; j < 3; j++) {
            p[j] = plaintext[i + j] - 'A';
        }

        for (j = 0; j < 3; j++) {
            c[j] = (key[j][0]*p[0] + key[j][1]*p[1] + key[j][2]*p[2]) % 26;
            printf("%c", c[j] + 'A');
        }
    }

    return 0;
}
```



Date: 09 / 01 / 2026

Output:

```
D:\CSE B-Tech\Semester-6\INS\Lab-6>gcc three_matrix.c  
  
D:\CSE B-Tech\Semester-6\INS\Lab-6>a  
Enter Plaintext (uppercase): ASK  
Ciphertext: AY0
```

Date: 07/02/2026

Lab Practical #7:

Implementation of rail fence cipher transposition techniques.

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char text[100], rail[10][100];
    int i, j, row = 0, dir = 1;
    int len, key;

    printf("Enter plaintext: ");
    scanf("%[^\n]", text);
    printf("Enter key: ");
    scanf("%d", &key);

    len = strlen(text);

    for (i = 0; i < key; i++){
        for (j = 0; j < len; j++){
            rail[i][j] = '\n';
        }
    }

    for (i = 0; i < len; i++) {
        rail[row][i] = text[i];
        if (row == 0){
            dir = 1;
        }
        else if (row == key - 1){
            dir = -1;
        }
        row = row + dir;
    }

    printf("Ciphertext: ");
    for (i = 0; i < key; i++){
        for (j = 0; j < len; j++){
            if (rail[i][j] != '\n'){
                printf("%c", rail[i][j]);
            }
        }
    }
}
```



Date: 07/02/2026

Output:

Enter plaintext: DARSHAN UNIVERSITY IS THE BEST

Enter key: 2

Ciphertext: DRHNUIEST STEBSASA NVRIYI H ET

Date: 07/02/2026

Lab Practical #8:

Implementation of row col cipher transposition techniques.

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char plain[200], text[200];
    char keystr[20];
    int key[20];
    int i, j, k = 0, cols, rows, len;

    printf("Enter key: ");
    scanf("%s", keystr);

    cols = strlen(keystr);

    for (i = 0; i < cols; i++){
        key[i] = keystr[i] - '0';
    }

    printf("Enter plaintext: ");
    getchar();
    gets(plain);

    for (i = 0; plain[i] != '\0'; i++) {
        if (plain[i] != ' ')
            text[k++] = plain[i];
    }

    text[k] = '\0';

    len = strlen(text);
    rows = len / cols;
    if (len % cols != 0){
        rows++;
    }
    char mat[rows][cols];

    k = 0;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (k < len){
                mat[i][j] = text[k++];
            }
        }
    }
}
```

Date: 07/02/2026

```
    }
    else{
        mat[i][j] = 'x';
    }
}

}

printf("\nKey: %s", keystr);
printf("\nPlaintext: %s", plain);
printf("\nCiphertext: ");

int count = 0;

for (i = 1; i <= cols; i++) {
    for (j = 0; j < cols; j++) {
        if (key[j] == i) {
            for (k = 0; k < rows; k++) {
                printf("%c", mat[k][j]);
                count++;
                if (count % cols == 0)
                    printf(" ");
            }
        }
    }
}
```

Output:

Key: 2341

Plaintext: puja n pipaliya

Ciphertext: apyx pnaa uplx jiix



Date: 13 / 02 / 2026

Lab Practical #9:

Implementation of Block Cipher Operation.

1. Electronic Code Book - ECB
2. Cipher Block Chaining Mode - CBC
3. Cipher Feedback Mode - CFB
4. Output Feedback Mode - OFB
5. Counter Mode - CTR

Program:

```
// 1. Electronic Code Book - ECB
// 1. Electronic Code Book - ECB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

void xorBlock(char *input, char *key, char *output)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        output[i] = input[i] ^ key[i];
}

void ECB(char *input, char *key, char *output, int length)
{
    for(int i = 0; i < length; i += BLOCK_SIZE)
        xorBlock(input + i, key, output + i);
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    ECB(plaintext, key, ciphertext, length);
    ciphertext[length] = '\0'; // Fix

    ECB(ciphertext, key, decrypted, length);
    decrypted[length] = '\0'; // Fix
```



Date: 13 / 02 / 2026

```
printf("\nEncrypted: %s", ciphertext);  
printf("\nDecrypted: %s", decrypted);  
  
return 0;  
}
```

Output:

```
Enter plaintext (multiple of 8): ABCDEFGH  
Encrypted: pppppppp  
Decrypted: ABCDEFGH
```


Program:

```
// 2. Cipher Block Chaining Mode - CBC

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// CBC Encryption
void CBC_Encrypt(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(input + i, iv, temp);
        xorBlock(temp, key, output + i);
        memcpy(iv, output + i, BLOCK_SIZE);
    }
}

// CBC Decryption
void CBC_Decrypt(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(input + i, key, temp);
        xorBlock(temp, iv, output + i);
        memcpy(iv, input + i, BLOCK_SIZE);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
```

Date: 13 / 02 / 2026

```
char original_iv[BLOCK_SIZE];

printf("Enter plaintext (multiple of 8): ");
scanf("%s", plaintext);

int length = strlen(plaintext);

// Save original IV
memcpy(original_iv, iv, BLOCK_SIZE);

// Encrypt
CBC_Encrypt(plaintext, key, iv, ciphertext, length);
ciphertext[length] = '\0'; // Null terminate

// Reset IV before decryption
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt
CBC_Decrypt(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\0'; // Null terminate

printf("\nEncrypted: %s", ciphertext);
printf("\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

Enter plaintext (multiple of 8): ABCDEFGH	
Encrypted: HGFEDCBA	
Decrypted: ABCDEFGH	

Program:

```
// 3. Cipher Feedback Mode - CFB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// CFB Mode (Same for Encrypt & Decrypt)
void CFB(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(iv, key, temp);           // Encrypt IV
        xorBlock(input + i, temp, output + i); // XOR with plaintext/ciphertext
        memcpy(iv, output + i, BLOCK_SIZE); // Update IV
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
    char original_iv[BLOCK_SIZE];

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    // Save original IV
    memcpy(original_iv, iv, BLOCK_SIZE);

    // Encrypt
```



Date: 13 / 02 / 2026

```
CFB(plaintext, key, iv, ciphertext, length);
ciphertext[length] = '\0';

// Reset IV before decryption
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt (same function)
CFB(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\0';

printf("\nEncrypted: %s", ciphertext);
printf("\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

```
Enter plaintext (multiple of 8): ABCDEDGH

Encrypted: HGFEDABA
Decrypted: ABCDEDGH
```

Program:

```
// 4. Output Feedback Mode - OFB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// OFB Mode (Same for Encrypt & Decrypt)
void OFB(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(iv, key, temp);
        memcpy(iv, temp, BLOCK_SIZE);
        xorBlock(input + i, iv, output + i);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
    char original_iv[BLOCK_SIZE];

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    memcpy(original_iv, iv, BLOCK_SIZE);

    // Encrypt
    OFB(plaintext, key, iv, ciphertext, length);
```



Date: 13 / 02 / 2026

```
ciphertext[length] = '\\0';

// Reset IV
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt (same function)
OFB(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\\0';

printf("\\nEncrypted: %s", ciphertext);
printf("\\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

```
Enter plaintext (multiple of 8): SDERFTCY

Encrypted: ZA@SGQFP
Decrypted: SDERFTCY
```

Program:

```
// 5. Counter Mode - CTR

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

void CTR(char *input, char *key, char *nonce, char *output, int length)
{
    char counterBlock[BLOCK_SIZE];
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        memcpy(counterBlock, nonce, BLOCK_SIZE);
        counterBlock[BLOCK_SIZE - 1] += i / BLOCK_SIZE;

        xorBlock(counterBlock, key, temp);
        xorBlock(input + i, temp, output + i);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char nonce[BLOCK_SIZE] = "00000000";

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    CTR(plaintext, key, nonce, ciphertext, length);
    CTR(ciphertext, key, nonce, decrypted, length);

    printf("\nEncrypted: %s", ciphertext);
}
```



Date: 13 / 02 / 2026

```
printf("\nDecrypted: %s", decrypted);  
  
return 0;  
}
```

Output:

```
Enter plaintext (multiple of 8): VXYUBI  
Encrypted: WZZQGO  
Decrypted: VXYUBI
```


Lab Practical #10:

1. Implementation of DES Algorithm.

Program:

```
// Implementation of DES Algorithm.

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define SIZE 100

/* ===== HEX ⇔ BIN ===== */

void hex2bin(char *hex, char *bin)
{
    char *map[16] = {
        "0000", "0001", "0010", "0011",
        "0100", "0101", "0110", "0111",
        "1000", "1001", "1010", "1011",
        "1100", "1101", "1110", "1111"
    };
    bin[0] = '\0';
    for(int i=0; hex[i]!='\0'; i++)
    {
        if(hex[i]>='0' && hex[i]<='9')
            strcat(bin, map[hex[i]-'0']);
        else
            strcat(bin, map[hex[i]-'A'+10]);
    }
}

void bin2hex(char *bin, char *hex)
{
    char *map[16] = {"0", "1", "2", "3", "4", "5", "6", "7",
        "8", "9", "A", "B", "C", "D", "E", "F"};
    hex[0] = '\0';
    for(int i=0; i<64; i+=4)
    {
        int num=0;
        for(int j=0; j<4; j++)
            num = num*2 + (bin[i+j]-'0');
        strcat(hex, map[num]);
    }
}
```

```
void xor(char *a, char *b, char *res)
{
    for(int i=0;a[i]!='\0';i++)
        res[i]=(a[i]==b[i])?'0':'1';
    res[strlen(a)]='\0';
}

void shift_left(char *key,int shifts)
{
    while(shifts--)
    {
        char first=key[0];
        for(int i=0;i<strlen(key)-1;i++)
            key[i]=key[i+1];
        key[strlen(key)-1]=first;
    }
}

void permute(char *in,int *table,int n,char *out)
{
    for(int i=0;i<n;i++)
        out[i]=in[table[i]-1];
    out[n]='\0';
}

/* ===== DES TABLES ===== */

/* Initial Permutation */
int initial_perm[64]={58,50,42,34,26,18,10,2,
60,52,44,36,28,20,12,4,
62,54,46,38,30,22,14,6,
64,56,48,40,32,24,16,8,
57,49,41,33,25,17,9,1,
59,51,43,35,27,19,11,3,
61,53,45,37,29,21,13,5,
63,55,47,39,31,23,15,7};

/* Final Permutation */
int final_perm[64]={40,8,48,16,56,24,64,32,
39,7,47,15,55,23,63,31,
38,6,46,14,54,22,62,30,
37,5,45,13,53,21,61,29,
36,4,44,12,52,20,60,28,
35,3,43,11,51,19,59,27,
34,2,42,10,50,18,58,26,
```



Date: 12/03/2026

```
33,1,41,9,49,17,57,25});

/* Expansion Table */
int exp_d[48]={32,1,2,3,4,5,4,5,
6,7,8,9,8,9,10,11,
12,13,12,13,14,15,16,17,
16,17,18,19,20,21,20,21,
22,23,24,25,24,25,26,27,
28,29,28,29,30,31,32,1};

/* Straight Permutation */
int per[32]={16,7,20,21,
29,12,28,17,
1,15,23,26,
5,18,31,10,
2,8,24,14,
32,27,3,9,
19,13,30,6,
22,11,4,25};

/* PC-1 */
int keyp[56]={57,49,41,33,25,17,9,
1,58,50,42,34,26,18,
10,2,59,51,43,35,27,
19,11,3,60,52,44,36,
63,55,47,39,31,23,15,
7,62,54,46,38,30,22,
14,6,61,53,45,37,29,
21,13,5,28,20,12,4};

/* PC-2 */
int key_comp[48]={14,17,11,24,1,5,
3,28,15,6,21,10,
23,19,12,4,26,8,
16,7,27,20,13,2,
41,52,31,37,47,55,
30,40,51,45,33,48,
44,49,39,56,34,53,
46,42,50,36,29,32};

/* Shift table */
int shift_table[16]={1,1,2,2,2,2,2,2,
1,2,2,2,2,2,2,1};

/* S-Box */
int sbox[8][4][16]={
```



Date: 12/03/2026

```
{ {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
  {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
  {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
  {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}},

{ {15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
  {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
  {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
  {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}},

{ {10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
  {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
  {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
  {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}},

{ {7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
  {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
  {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
  {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}},

{ {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
  {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
  {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
  {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}},

{ {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
  {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
  {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
  {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}},

{ {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
  {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
  {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
  {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}},

{ {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
  {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
  {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
  {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}}
};

/* ===== MAIN ===== */

/* ===== MAIN ===== */

int main()
```

```
{
    char pt[20], key[20];
    char pt_bin[100], key_bin[100];

    printf("Enter 16 Hex Plaintext: ");
    scanf("%s",pt);

    printf("Enter 16 Hex Key: ");
    scanf("%s",key);

    printf("\nOriginal Plaintext: %s\n", pt);

    hex2bin(pt,pt_bin);
    hex2bin(key,key_bin);

    /* ---- Key Generation ---- */
    char key56[100];
    permute(key_bin,keyp,56,key56);

    char left[30],right[30];
    strncpy(left,key56,28); left[28]='\0';
    strncpy(right,key56+28,28); right[28]='\0';

    char round_key[16][50];

    for(int i=0;i<16;i++)
    {
        shift_left(left,shift_table[i]);
        shift_left(right,shift_table[i]);

        char combine[60];
        strcpy(combine,left);
        strcat(combine,right);

        permute(combine,key_comp,48,round_key[i]);
    }

    /* ===== ENCRYPTION ===== */

    char ip[100];
    permute(pt_bin,initial_perm,64,ip);

    char L[40],R[40];
    strncpy(L,ip,32); L[32]='\0';
    strncpy(R,ip+32,32); R[32]='\0';
```

```
for(int i=0;i<16;i++)
{
    char R_exp[60];
    permute(R,exp_d,48,R_exp);

    char xored[60];
    xor(R_exp,round_key[i],xored);

    char sbbox_str[40]="";

    for(int j=0;j<8;j++)
    {
        int row = (xored[j*6]-'0')*2 + (xored[j*6+5]-'0');
        int col = (xored[j*6+1]-'0')*8 +
                  (xored[j*6+2]-'0')*4 +
                  (xored[j*6+3]-'0')*2 +
                  (xored[j*6+4]-'0');

        int val = sbbox[j][row][col];

        char bin[5];
        for(int k=3;k>=0;k--)
            bin[3-k] = ((val>>k)&1)+'0';

        bin[4]='\0';
        strcat(sbbox_str,bin);
    }

    char f_out[40];
    permute(sbbox_str,per,32,f_out);

    char newR[40];
    xor(L,f_out,newR);

    strcpy(L,R);
    strcpy(R,newR);
}

char combine[100];
strcpy(combine,R);
strcat(combine,L);

char cipher_bin[100];
permute(combine,final_perm,64,cipher_bin);

char cipher_hex[20];
```

```
bin2hex(cipher_bin,cipher_hex);

printf("\nEncrypted Cipher Text: %s\n",cipher_hex);

/* ===== DECRYPTION ===== */

char cipher_bin_input[100];
hex2bin(cipher_hex,cipher_bin_input);

char ip_dec[100];
permute(cipher_bin_input,initial_perm,64,ip_dec);

char Ld[40],Rd[40];
strncpy(Ld,ip_dec,32); Ld[32]='\0';
strncpy(Rd,ip_dec+32,32); Rd[32]='\0';

for(int i=15;i>=0;i--)
{
    char R_exp[60];
    permute(Rd,exp_d,48,R_exp);

    char xored[60];
    xor(R_exp,round_key[i],xored);

    char sbbox_str[40]="";

    for(int j=0;j<8;j++)
    {
        int row = (xored[j*6]-'0')*2 + (xored[j*6+5]-'0');
        int col = (xored[j*6+1]-'0')*8 +
                  (xored[j*6+2]-'0')*4 +
                  (xored[j*6+3]-'0')*2 +
                  (xored[j*6+4]-'0');

        int val = sbbox[j][row][col];

        char bin[5];
        for(int k=3;k>=0;k--)
            bin[3-k] = ((val>>k)&1)+'0';

        bin[4]='\0';
        strcat(sbbox_str,bin);
    }

    char f_out[40];
    permute(sbbox_str,per,32,f_out);
```

Date: 12/03/2026

```
char newR[40];
xor(Ld,f_out,newR);

strcpy(Ld,Rd);
strcpy(Rd,newR);
}

char combine_dec[100];
strcpy(combine_dec,Rd);
strcat(combine_dec,Ld);

char plain_bin[100];
permute(combine_dec,final_perm,64,plain_bin);

char plain_hex[20];
bin2hex(plain_bin,plain_hex);

printf("Decrypted Text: %s\n",plain_hex);

return 0;
}
```

Output:

```
Enter 16 Hex Plaintext: 123456ABCD132536
Enter 16 Hex Key: AABB09182736CCDD

Original Plaintext: 123456ABCD132536

Encrypted Cipher Text: C0B7A8D05F3A829C
Decrypted Text: 123456ABCD132536
```


Date: 12/03/2026

Lab Practical #12:

Implementation of key modular arithmetic operations in C.

1. Write a C program to find GCD of two numbers using the Euclidean algorithm.
2. Write a C program to implement modular exponentiation (for RSA key generation).
3. Write a C program to find the multiplicative inverse of a number modulo n.

Program:

```
// 1. Write a C program to find GCD of two numbers using the Euclidean algorithm.

#include <stdio.h>

int gcd(int a, int b)
{
    while(b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("GCD = %d\n", gcd(a, b));

    return 0;
}
```

Output:

```
Enter two numbers: 48 18
GCD = 6
```



Date: 12/03/2026

Program:

```
// 2. Write a C program to implement modular exponentiation (for RSA key generation).

#include <stdio.h>

long long modExp(long long base, long long exp, long long mod)
{
    long long result = 1;

    while(exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}

int main()
{
    long long base, exp, mod;

    printf("Enter base exponent modulus: ");
    scanf("%lld %lld %lld", &base, &exp, &mod);

    printf("Result = %lld\n", modExp(base, exp, mod));

    return 0;
}
```

Output:

```
Enter base exponent modulus: 5 3 13
Result = 8
```

Date: 12/03/2026

Program:

```
// 3. Write a C program to find the multiplicative inverse of a number modulo n.

#include <stdio.h>

int modInverse(int a, int n)
{
    int x;

    for(x = 1; x < n; x++)
    {
        if((a * x) % n == 1)
            return x;
    }

    return -1;
}

int main()
{
    int a, n;

    printf("Enter number and modulo: ");
    scanf("%d %d", &a, &n);

    int inv = modInverse(a, n);

    if(inv == -1)
        printf("Inverse does not exist\n");
    else
        printf("Multiplicative inverse = %d\n", inv);

    return 0;
}
```

Output:

```
Enter number and modulo: 3 11
Multiplicative inverse = 4
```

Date: 13/03/2026

Lab Practical #10:

1. Implementation of AES Algorithm.

Program:

```
// 1. Implementation of AES Algorithm.

#include <stdio.h>

#define Nr 10

char state[4][4];

// Simple S-box (short version for lab)
char sbox[16] = {0x6, 0x4, 0xC, 0x5, 0x0, 0x7, 0x2, 0xE, 0x1, 0xF, 0x3, 0xD, 0x8, 0xA, 0x9, 0xB};

void AddRoundKey(char key[4][4])
{
    for(int i=0; i<4; i++)
        for(int j=0; j<4; j++)
            state[i][j] ^= key[i][j];
}

void SubBytes()
{
    for(int i=0; i<4; i++)
        for(int j=0; j<4; j++)
            state[i][j] = sbox[state[i][j] % 16];
}

void ShiftRows()
{
    char temp;

    temp = state[1][0];
    state[1][0] = state[1][1];
    state[1][1] = state[1][2];
    state[1][2] = state[1][3];
    state[1][3] = temp;

    char t1 = state[2][0];
    char t2 = state[2][1];
    state[2][0] = state[2][2];
    state[2][1] = state[2][3];
    state[2][2] = t1;
    state[2][3] = t2;
}
```

```
temp = state[3][3];
state[3][3]=state[3][2];
state[3][2]=state[3][1];
state[3][1]=state[3][0];
state[3][0]=temp;
}

void MixColumns()
{
    for(int i=0;i<4;i++)
    {
        char t = state[0][i]^state[1][i]^state[2][i]^state[3][i];
        state[0][i]^=t;
        state[1][i]^=t;
        state[2][i]^=t;
        state[3][i]^=t;
    }
}

void printState()
{
    for(int i=0;i<4;i++)
    {
        for(int j=0;j<4;j++)
            printf("%d ",state[i][j]);
        printf("\n");
    }
}

int main()
{
    char plaintext[4][4];
    char key[4][4];

    printf("Enter 16 plaintext values:\n");
    for(int i=0;i<4;i++)
        for(int j=0;j<4;j++)
            scanf("%d",&plaintext[i][j]);

    printf("Enter 16 key values:\n");
    for(int i=0;i<4;i++)
        for(int j=0;j<4;j++)
            scanf("%d",&key[i][j]);

    for(int i=0;i<4;i++)
```

Date: 13/03/2026

```
for(int j=0;j<4;j++)
    state[i][j]=plaintext[i][j];

AddRoundKey(key);

for(int round=1; round<Nr; round++)
{
    SubBytes();
    ShiftRows();
    MixColumns();
    AddRoundKey(key);
}

SubBytes();
ShiftRows();
AddRoundKey(key);

printf("\nCiphertext:\n");
printStats();

return 0;
}
```

Output:

```
Enter 16 plaintext values:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
Enter 16 key values:
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1

Ciphertext:
14 7 2 2
3 11 8 9
0 4 11 2
14 1 4 5
```

Date: 14/03/2026

Lab Practical #13:

Implementation of RSA Algorithm Technique.

Program:

```
// Implementation of RSA Algorithm Technique.

#include <stdio.h>
#include <math.h>

// Function to find GCD
int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Function to find (base^exp) % mod
long long modExp(long long base, long long exp, long long mod)
{
    long long result = 1;

    while (exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}

int main()
{
    int p, q, n, phi, e, d = 1, k = 1;
    int msg;
    long long cipher, decrypt;

    printf("Enter two prime numbers (p and q): ");
    scanf("%d %d", &p, &q);

    n = p * q;
```

Date: 14/03/2026

```
phi = (p - 1) * (q - 1);

printf("Enter value of e: ");
scanf("%d", &e);

while (gcd(e, phi) != 1)
{
    printf("e must be coprime with %d. Enter again: ", phi);
    scanf("%d", &e);
}

// Finding d
while (1)
{
    if ((k * phi + 1) % e == 0)
    {
        d = (k * phi + 1) / e;
        break;
    }
    k++;
}

printf("Public Key (e, n) = (%d, %d)\n", e, n);
printf("Private Key (d, n) = (%d, %d)\n", d, n);

printf("Enter message (number): ");
scanf("%d", &msg);

cipher = modExp(msg, e, n);
printf("Encrypted Message = %lld\n", cipher);

decrypt = modExp(cipher, d, n);
printf("Decrypted Message = %lld\n", decrypt);

return 0;
}
```




Date: 14/03/2026

Output:

```
Enter two prime numbers (p and q): 3 11
Enter value of e: 3
Public Key (e, n) = (3, 33)
Private Key (d, n) = (7, 33)
Enter message (number): 7
Encrypted Message = 13
Decrypted Message = 7
```

Date: 14/03/2026

Lab Practical #14:

Implementation of Diffie Hellman Key exchange technique.

Program:

```
// Implementation of Diffie-Hellman Key Exchange Technique

#include <stdio.h>
#include <math.h>

// function for modular exponentiation
long long power(long long base, long long exp, long long mod)
{
    long long result = 1;

    for(int i = 0; i < exp; i++)
    {
        result = (result * base) % mod;
    }

    return result;
}

int main()
{
    long long p, g;
    long long a, b;
    long long A, B;
    long long keyA, keyB;

    printf("Enter prime number (p): ");
    scanf("%lld", &p);

    printf("Enter primitive root (g): ");
    scanf("%lld", &g);

    printf("Enter private key of User A: ");
    scanf("%lld", &a);

    printf("Enter private key of User B: ");
    scanf("%lld", &b);

    // Public keys
    A = power(g, a, p);
    B = power(g, b, p);

    printf("\nPublic key of User A = %lld", A);
```



Date: 14/03/2026

```
printf("\nPublic key of User B = %lld", B);

// Secret keys
keyA = power(B, a, p);
keyB = power(A, b, p);

printf("\n\nSecret key calculated by User A = %lld", keyA);
printf("\nSecret key calculated by User B = %lld", keyB);

return 0;
}
```

Output:

```
Enter prime number (p): 23
Enter primitive root (g): 5
Enter private key of User A: 6
Enter private key of User B: 15

Public key of User A = 8
Public key of User B = 19

Secret key calculated by User A = 2
Secret key calculated by User B = 2
```

Date: 14/03/2026

Lab Practical #15:

To demonstrate basic data integrity techniques in C by implementing checksum calculation, simple hashing, and CRC generation.

Program:

```
// Demonstration of Data Integrity Techniques
// Checksum, Simple Hashing and CRC

#include <stdio.h>
#include <string.h>

/* ----- CHECKSUM ----- */
int checksum(char data[])
{
    int sum = 0;

    for(int i = 0; data[i] != '\0'; i++)
    {
        sum = sum + data[i];
    }

    return sum % 256;    // simple checksum
}

/* ----- SIMPLE HASH ----- */
int simpleHash(char data[])
{
    int hash = 0;

    for(int i = 0; data[i] != '\0'; i++)
    {
        hash = (hash * 31 + data[i]) % 1000;
    }

    return hash;
}

/* ----- CRC ----- */
void crc(char data[], char key[])
{
    char temp[100], remainder[100];
    int dataLen = strlen(data);
    int keyLen = strlen(key);

    strcpy(temp, data);
```

Date: 14/03/2026

```
for(int i = 0; i < keyLen - 1; i++)
    temp[dataLen + i] = '0';

temp[dataLen + keyLen - 1] = '\\0';

strcpy(remainder, temp);

for(int i = 0; i <= strlen(temp) - keyLen; i++)
{
    if(remainder[i] == '1')
    {
        for(int j = 0; j < keyLen; j++)
        {
            remainder[i+j] =
                (remainder[i+j] == key[j]) ? '0' : '1';
        }
    }
}

printf("CRC Remainder: ");
for(int i = strlen(temp) - keyLen + 1; i < strlen(temp); i++)
    printf("%c", remainder[i]);
}

/* ----- MAIN ----- */
int main()
{
    char data[100];
    char key[20];

    printf("Enter data: ");
    scanf("%s", data);

    printf("Enter CRC key (binary): ");
    scanf("%s", key);

    printf("\nChecksum value: %d\n", checksum(data));
    printf("Simple Hash value: %d\n", simpleHash(data));

    crc(data, key);

    return 0;
}
```



Date: 14/03/2026

Output:

```
Enter data: hello
Enter CRC key (binary): 1011

Checksum value: 20
Simple Hash value: 322
CRC Remainder: 000
```